

Indexing in DBMS — Basic Introduction

Database Management Systems — Architecture, I/O Cost & Core Concepts

1. What is Indexing?

Indexing is a data structure technique used in DBMS to optimize search efficiency by minimizing the number of disk block accesses required when executing a query.

🎯 Main Goal

Reduce Disk Blocks Accessed
→ Reduce I/O Cost
→ Accelerate Query Execution

💡 Example Query

```
SELECT * FROM Student
```

```
WHERE Roll_No = 1;
```

Locates student record without scanning all table blocks.

⚠️ **CRITICAL EXAM TRAP:** Indexing **does NOT** reduce the time required to search *inside* an individual block. Its primary function is reducing the *number of blocks* brought from secondary storage into RAM.

2. Basic Architecture & Data Flow

1. CPU

Executes/processes incoming SQL queries. Extremely fast compared to storage.

2. RAM

Volatile working memory. Blocks are transferred here from disk for processing.

3. Secondary Storage

Hard Drives / SSDs. Persistent storage divided into fixed-size **blocks/pages**.



3. Numerical Example: Block Calculation

Consider a dataset with **10,000 records** where each disk block stores **100 records**:

$$\begin{aligned}\text{Number of Blocks Required} &= \text{Total Records} / \text{Records per Block} \\ &= 10,000 / 100 = \mathbf{100 \text{ Blocks}}\end{aligned}$$

4. I/O Cost, Hits & Misses

Input / Output (I/O) Cost

- Transferring blocks between Secondary Storage and RAM incurs I/O cost.
- More Blocks Accessed** = Higher I/O Cost.
- Fewer Blocks Accessed** = Lower I/O Cost.

Hit vs. Miss

- Hit** ✅: Target record is present in the block fetched into RAM.
- Miss** ❌: Target record is not present; another block must be fetched.

5. Book Analogy vs. Database Index

Understanding indexing through the concept of a textbook index:

Feature	Without Index (Linear Scan)	With Index
Textbook Search	Scanning up to 1,000 pages line-by-line (Worst: 1,000, Avg: ~500 pages).	Look up topic in index (2 pages) → Jump directly to Page 500.
Database Search	Fetch Block 1 → RAM → Check ❌ → Fetch Block 2 → RAM → Check ❌...	Lookup Index → Identify exact Block ID → Fetch only target Block ✅.
I/O Overhead	High (Scans many blocks).	Extremely Low (Scans minimal blocks).

★ GOLDEN RULE OF INDEXING:

INDEX → Blocks Accessed ↓ → I/O Operations ↓ → Cost ↓ → Performance ⚡

6. Quick Revision Table

Concept	Meaning & Significance
CPU	Processes SQL queries and handles logical processing.
RAM	Fast, volatile working memory; holds active blocks.
Secondary Storage	Persistent storage (HDD/SSD) holding database files.
Block / Page	Fixed-size storage unit containing a set of records.
I/O Cost	Performance penalty incurred when reading/writing blocks to/from disk.
Primary Purpose	Reduce total block access to lower overall I/O cost.

7. Frequently Tested Exam Questions (Exam Traps)

Q1. What is the primary purpose of creating an index in a DBMS?

Ans: To reduce the number of disk blocks accessed during a search, thereby lowering I/O cost.

Q2. Does indexing speed up the search time within an individual block?

Ans: No. It reduces the number of blocks transferred to RAM, not internal block search time.

Q3. Why is persistent database data stored in secondary storage instead of RAM?

Ans: Secondary storage provides non-volatile (permanent) storage, whereas RAM is volatile and loses data upon power loss.

Q4. How does block access count relate to I/O cost?

Ans: More block accesses directly result in higher I/O operations and cost; fewer block accesses mean lower I/O cost.